

review

Review — Workshop Curriculum Platform

Date: 2026-09-02 · **Feature:** specs/001-workshop-curriculum-platform
Phase: review — first execution. analyze ran once (2026-09-01) against a **91-task** revision of tasks.md; implement has never been executed as a tracked SpecKit phase, so every line of the platform was written by hand outside it. This document is the reconciliation of the current **120** tasks against the tree, plus the review findings the phase exists to produce.

Scope note. This repository is PUBLIC. No transcript text, participant name or recording filename appears anywhere below. Where a defect concerns such data, the defect is described and the data is not.

0. Why this document had to be written

tasks.md carried **120 T### identifiers and 0 ticked checkboxes**, and .superpowers/sdd/progress.md listed zero completed tasks — while the tree held a running service on the LAN with 30+ registered routes, 1 101 minted passages, a cross-reference graph, an entailment verifier, a 57-question answering benchmark and a reworked Angular client. Nothing had ever reconciled the two. The gap was not that work was missing; it was that **nobody could tell which work was missing**, and that is the condition in which a decorative requirement survives.

After reconciliation: **54 of 120 tasks are now ticked**, and the 66 that are not are enumerated with a reason each.

1. The evidence standard applied

A task was ticked only against an artifact that exists and behaves. Concretely:

1. **A file on disk, named.** Not a plan sentence, not a comment claiming the thing, not a document asserting it. Every BUILT verdict below cites a path, a route, a test function or a gate.
2. **A gate named in the task is part of the deliverable.** Standing rule 1 of `tasks.md` makes a paired mutation proof a condition of a check existing at all. So a task of the shape “*implement X · Gate G-N: ... · Paired mutation: ...*” is ticked only when **both** halves exist. Nine tasks were withheld on exactly this basis — T019, T021 and T029; then T059, T073 and T075; then T078, T081 and T114 — the module is there, the proof is not.
3. **The live service outranks the manifest.** `workshop/platform/gates/route-manifest.tsv` is a hand-maintained declaration. Every one of its 25 rows was probed against `192.168.1.44:8087` rather than read.
4. **Content type discriminates a 404.** `404 application/json` with a contract body means the route exists and is answering “no” (measured: `chapter_not_found`, `material_not_found`, `job_not_found`, `not_found` for an absent pid). `404 text/plain` – 404 page not found means the route is not registered (measured: `/api/nosuchroute` on all four verbs, `/api/crossrefs`, `/api/knowledge`). Grading on the status code alone would have mis-filed at least six endpoints here.
5. **Repository documentation is not evidence.** Several figures in the carriers and in `tasks.md` are measurably stale — see §6. Every number below was re-derived.
6. **Could-not-determine is a result.** Two tasks are recorded as such rather than guessed.

A “BUILT DIFFERENTLY” verdict was ticked only when the intent is satisfied and the difference does not matter. Thirteen were. Twenty more are recorded as BUILT DIFFERENTLY and left **unticked**, because the difference does matter — those are review findings, not bookkeeping.

2. Disposition — all 120 tasks

Verdict	Tasks	Ticked
BUILT — present where the task	41	41

Verdict	Tasks	Ticked
says, shaped as the task says		
BUILT DIFFERENTLY — the intent is met and the divergence costs nothing	13	13
BUILT DIFFERENTLY — the intent falls short, or a gate the task names is missing	20	0
NOT BUILT	44	0
SUPERSEDED by a later recorded decision	0	—
CANNOT DETERMINE	2	0
Total	120	54

Re-derive the tick count rather than trusting this table:

```
cd specs/001-workshop-curriculum-platform
grep -c '^-\[x\] T' tasks.md # 54 ticked
grep -c '^-\[ \] T' tasks.md # 66 not ticked
```

2.1 BUILT — 41, ticked

Setup — T001 (workshop/platform/{backend,frontend,bin} + workshop/{curriculum,pipeline,docs,evidence}, and no workshop/containers/) · T002 (platform/backend/go.mod, go 1.26.2) · T003 (the module requires digital.vasic.containers at v0.0.0, plus replace ... => ../../../../submodules/containers, and go test ./... passes) · T004 (Angular 19.2, karma/jasmine, deps installed) · T007 (measured: git -C workshop ls-files | grep '^\.github/workflows/.*\ya?ml\$' is empty; gate-E essence re-run across all 13 declared gitlinks — workshop = 0).

Foundational — T008 submodules/passage/pkg/passage/pid.go (ULID minter, monotonic across clock stall, deliberately no Time() accessor) · T009 anchor.go (HTML/Mermaid/PlantUML syntaxes, ScanAnchors

rejects duplicate and dangling anchors) · T010 registry.go
(passages.jsonl → passages.db; content_hash in no PK, unique index or FK) · T018 submodules/verdict/pkg/verdict/verdict.go (0/1/2, Tally precedence Problem > Undetermined > OK, **empty tally** = **Undetermined**, JSON emits the name never the integer) · T108 scripts/build.sh · T109 scripts/restart.sh (stop→start with 1/2 short-circuit, unclassified → 2) · T110 **G-CLI-16** — scripts/verify.sh V4 gate_lifecycle:298-306 with the exact paired mutation the task names (podman-compose up -d fallback seeded, :558-559 ⇒ rc 1), a variable-held-runtime case (V4c), and a read-only podman ps control (V4b) that *shows* the exemption rather than asserting it.

US1 — T112 scripts/verify-accuracy.sh (requires --reference; **exit 2 when it is absent**, :665/:669; --windows/--seed/--normaliser/--min-accuracy; writes accuracy.json; **G-CLI-5** named at :88, :395, :452, :544, :554 with a built-in --selftest that seeds a bluff into the real guard block). Verified live: GET /api/chapters/01/accuracy → {"measured": false, "wer": null, "reason": ..., "remedy": ...} — exactly the contracted shape for “not measured”.

US2 — T043 / T044 (/api/chapters, /api/chapters/{slug}, /api/chapters/{slug}/transcript — all probed live) · T046 / T047 (features/chapters/, features/transcript/ exist and are routed) · T050 (features/transcript/seek.spec.ts:139-141 and :219-220 assert seek publish < 5 000 ms and video.currentTime to 3 decimal places).

US3 — T052 (FTS5 prefix, pkg/search/lexical.go, AND p.redacted = 0) · T053 **G-HTTP-8** (internal/api/gates_test.go:286 — a real net.Listen counting *accepted connections*, 200 suggest calls + 25 forced 503s, asserts conns.Load() == 0; live /api/suggest returns legs: {"lexical": "ok"} and nothing else) · T054 · T055 **G-HTTP-1/2/3** (pkg/search/envelope.go:302-510; verified live — a partial failure returns **503 partial_failure_zero_results** with **no results key at all**) · T056 **G-HTTP-4** (degraded.go:42-52, 61-75, 125-134; the glued upstream string is confined to degraded.evidence) · T057 **G-HTTP-7** (search.component.spec.ts:23 — both directions, plus a paired-mutation control at :186-216 that reimplements error: () => loading.set(false) and proves the gate has teeth) · T058 **G-IDX-1** (verify-before-swap in one transaction, pkg/index/generation.go:141-167) · T060 / T061 (cycle-safe traversal, redacted_omitted as a count that never names a pid, *int without omitempty so “not traversed” is explicit null) · T062 (exhaustive @switch over ok|no_match|unavailable plus two further distinct states) · T066 **G-PID-3** (pid_test.go:465-508, including the mutation that keys a citation on content_hash and watches it dangle) · T117 **G-HTTP-6**

(`gates_test.go:227`, 100 randomised outcomes, and it **fails if the run never produced each outcome**; live header `X-Workshop-Search-Status` matched body status on every probe).

US4 — `T068` (four adapters: `"/none`, `extractive`, `ollama`, `openai_compatible` behind an explicit `AllowRemote`; unknown name → `UnavailableProvider`, never a panic) · `T069` · `T070` (**both** `MinScore` and `MinMargin`; live `/api/ask/status` reports `min_score 0.45`, `min_margin 0.05`, `calibrated true`) · `T071` (`schema.go:115` `minItems:1` on citations **and** `:82` on claims) · `T072` (`L3a` in-request membership + `L3b` `registry.Resolve` against the server's own loaded registry, four outcomes kept apart) · `T074` (whole-answer refusal, `verify.go:127-131,435-445,522-531`, `pipeline.go:303-323`) · `T077` (`testdata/benchmark/questions.tsv` — **24 A rows and 33 U rows**, measured) · `T082` (`ingest.sh --require-suspend` → `POST /api/ask/suspend|resume`; 5 tests; live `suspended: false`).

Polish — `T091 docs/user-guide.md` · `T092 docs/manual.md` · `T093 docs/faq.md` · `T100 G-CLI-11` (`verify.sh V5` `gate_workflows:367` with three paired cases — workflow seeded ⇒ 1, nested checkout ⇒ 0, empty tree ⇒ 0).

2.2 BUILT DIFFERENTLY, intent met — 13, ticked

Task	Planned	Actual	Why the difference is immaterial
T011	passage/ resolve.go	registry.go:295-309, 462-478	All four outcomes present and distinct; undetermined is carried by an unavailable error field, never collapsed; no fuzzy, nearest-neighbour, prefix or <code>content_hash</code> fallback exists. Only the filename differs. Assertions match clause for clause, both paired mutations are injected through the real <code>SyncFi</code> via the <code>real</code> <code>WithKeyStrategy</code> , and a third test proves the two mutants are <i>discriminating</i> (<code>content-hash</code> keying survives <code>G</code> positional keying survives <code>G1</code>). Only the identifiers differ — which matters solely to the gate-
T013	gates G-PID-1, G-PID-2	pid_test.go gates G1, G2	

Task	Planned	Actual	Why the difference is immaterial
			coverage grep in tasks.md §“Gate coverage”.
T024	transcribe/ vad.py	pipeline/ audio_energy.py:65-93	Produces the non-speech span so the task needs ({from_s,to_s,seconds,mean_db},p5 noise floor, ≤ floor+6 dB). Energy-based rather than a VAD model; output exists at pipeline/work/full_ch01.energy.json.
T026	transcribe/ asr.py	pipeline/ run_faster_whisper.py:107	condition_on_previous_text=False; present; the second engine mirrors it with -mc 0 (run_whispercpp.sh:75).
T027	transcribe/ confidence.py	pipeline/ build_transcript.py:198-209	Marks rather than guesses. Uses mean word probability + no_speech_prob; avg_logprob is passed through but not decisive. Different signal, same guarantee
T033	accuracy/ score.py	scripts/verify- accuracy.sh:201-262	≥30 stratified 30-second audio-timeline windows, seeded and recorded, oversampled then de-overlapped. The bias the task warns about (sampling passages hides whole-region deletions) is named in the code.
T036	300 s calibration	pipeline/calibrate.sh + calibration/sample_1800.* + CALIBRATION.md	A two-engine calibration ran, on longer sample than asked. Whether it settles research item U1–U3 specifically was not verified here.
T042	internal/ store/ chapter.go	internal/api/chapters.go + pkg/curriculum	A read-only filesystem store rather than a database. Legitimate for a read-only curriculum; the endpoints serve rich chapter object.
T045	pkg/media/ serve.go	internal/api/ recording.go:786	http.ServeContent with a real integrity sidecar gate. Verified live: Range: bytes=0-99 → 206 with Content-Range: bytes 0-99/1871981557; HEAD → 200 with

Task	Planned	Actual	Why the difference is immaterial
T048	reuse design-system/ learning-kit/	vendored copy + sync-learning-kit.sh --check	Accept-Ranges: bytes; multi-range refused with 416. diff -q and SHA-256 both show the two files byte-identical today. Vendored deliberately so the module stays independently cloneable, with a drift check wired into npm run.
T089	docs/add-chapter-prompt.md	docs/prompts/add-a-chapter.md	Same artifact, filed under a prompts/ directory.
T094	docs/ training.md + docs/ tutorial-quickstart.md	docs/training/00-Overview-and-Taxonomy.md + docs/training/areas/01..05 + docs/quickstart.md	A training set of six documents rather than one file — a superse
T108	build.sh incl. container images	scripts/build.sh:73-115	Builds workshop-boot, the Go server and the Angular bundle in order. It builds no image , because compose.yml:78 runs a stock alpine:3.20 with host-built binaries bind-mounted and declares no build: key. The §11.4.76(4) obligation is honoured by there being no image build at all, not violated — and no direct podman build/docker build exists anywhere.

2.3 BUILT DIFFERENTLY, intent NOT met — 20, deliberately unticked

These are the review's substantive findings. Each is a task, not a patch.

Task	What exists	What is missing, and why it matters
T019	pipeline/detect_media.sh probes real capability (-show_format -show_streams, then asserts ^duration= and	The exit semantics are inverted. T019 requires an unusable tool to exit 2 ("an unusable tool determines

Task	What exists	What is missing, and why it matters
	^codec_type=audio), never --version; has --prove-failure over healthy/broken/silent fixtures.	nothing”). The script exits 1 for UNUSABLE and reserves 2 for “the probe reached no verdict” (:88-93, :425). Both readings are defensible; they are not the same, and only one can be right. G-CLI-3 appears nowhere in the tree. This needs an operator ruling, not a patch.
T021	scripts/ {start,stop,status}.sh drive workshop-boot, whose source (platform/orchestration/cmd/workshop-boot/main.go) consumes the containers submodule’s Go API (pkg/compose, pkg/health, pkg/runtime) and constructs the manager with this project’s compose file. It does not wrap cmd/boot. No lifecycle path invokes podman/docker/compose.	G-CLI-12 does not exist — nothing binds the published port with an unidentifiable holder and asserts status.sh exits 2. A different gate was built instead (G-CLI-17, see §5), which is good work but is not this one.
T029	The coverage identity is implemented — pkg/curriculum/curriculum.go:389 ComputeCoverage, with a documented coverageEpsilon = 1e-6 for float error only.	Two things. (a) It lives in Go, not transcribe/coverage.py, and G-CLI-4 does not exist — nothing deletes a chunk’s output and asserts exit 1. (b) It is currently reporting a failure and nothing acts on it — see §4.3.
T030	curriculum/chapter-01/transcript.md exists with timestamps and provenance.	It carries zero pid anchors. Measured: grep -c 'pid:' → 0 on both transcript.md and exercise-01.md, and grep -nE '^<!--' finds no comment lines at all. Meanwhile every one of the 1 101 registry rows

Task	What exists	What is missing, and why it matters
T031	The immutable machine layer is real: <code>registry.go:501-503</code> refuses any mutation of <code>machine_text</code> , and Provenance transitions one way.	declares <code>source_ref.anchor</code> = "inline". See §4.4. There is no append-only correction overlay. A correction overwrites text in place and flips provenance. <code>pkg/transcript/layers.go</code> does not exist. Correction <i>history</i> is therefore not retained — only the before (machine) and the current, never the sequence. It is not frozen. The hash is enforced only when the caller passes <code>--normaliser-sha256</code>; no pinned expected value is stored anywhere. “Recorded” is not “frozen”, and the task’s whole point is that the arithmetic is fixed <i>before</i> the first measurement. Two stores that never meet. <code>core/progress.ts</code> writes <code>localStorage</code> only (<code>KEY_PREFIX = 'workshop-progress:'</code>), while <code>features/progress/progress.component.ts:190</code> reads the server route, which is registered (<code>main.go:303-304</code>) and live (<code>GET /api/progress</code> → 400 X-Session is required). What the reader writes, the progress page never sees. The header comment in <code>progress.ts</code> justifying <code>localStorage</code> says <code>/api/</code>
T034	The normaliser (<code>compare_engines.py:52-56</code> <code>norm_word + align</code>) is imported rather than reimplemented, and its SHA-256 is recorded on every run.	
T049	Progress is persisted, twice.	

Task	What exists	What is missing, and why it matters
T051	docs/quickstart.md exists and is substantial.	progress was measured 404 — that claim is now stale. No timed fresh-clone run is recorded. Its line 3 asserts a fifteen-minute path from a fresh clone to a working search (paraphrased — that file is in the PRIVATE submodule, so it is cited by path, not quoted); no stopwatch figure for the whole run appears in the file. SC-004 is claimed, not measured. Aggravating factor: pipeline/venv-setup.sh does not exist (T005), so a fresh clone cannot reproduce the Python environment at all. Two independent mechanisms, one gate. The embedding exclusion runs in a <i>separate background pass</i> (startVectorIndexing), not inside index.Build, and the generation verification gate checks FTS membership only, never the vector set. R5 (“report degraded until the post-redaction rebuild completes”) is not evidenced anywhere. Since nothing can <i>perform</i> a redaction (T012, T038), none of this has ever been exercised end to end.
T059	Redacted pids are excluded from FTS at build time (registry.go:1075) and from the embedding set (semantic.go:270), with a read-side re-filter.	
T063	A real a11y surface: @axe-core/playwright scans per route in both themes (e2e/ally-responsive.spec.ts), ARIA 1.2 combobox keyboard	The suite has never passed. e2e/artifacts/results.json (2026-09-01T21:21Z) records expected: 20, unexpected:

Task	What exists	What is missing, and why it matters
	handling with aria-activedescendant (search.component.ts:392-434), unit-level a11y assertions, and contrast work documented with measured axe findings in styles.scss:78-125.	58, every failure net::ERR_CONNECTION_REFUSED — the server was not up. Written, not proven. SC-017 is unmeasured.
T073	L4 exists and refuses honestly on undecidable input.	It is not what the task specifies, and on the running server it is weaker still. (a) Floor 1 is lexical content-word overlap (verify.go:234-260), not an embedding floor — the task's own sentence is quoted in the code at :214-215 beside a different implementation. (b) Entailment runs one pair at a time (verify.go:487-512); there is no Batch method in pkg/entail. (c) main.go:475-494 calls answering.Wire without EntailPython/EntailModelDir, so per wire.go:141-147 the deployed server falls back to the lexical floor. The live service says so itself. See §4.2. Stored answers are never marked withdrawn — because jobs are in-memory and non-durable by design (jobs.go:12-17), so there are no stored answers. That may be the right design, but it is a silent change to the requirement, recorded nowhere.
T075	The decline half is built and tested: a citation resolving redacted produces declined{redacted_evidence} for the whole answer (pipeline_test.go:256).	
T078		

Task	What exists	What is missing, and why it matters
	platform/backend/gates/bench-answers.sh exists, classifies each question, is three-valued, and records retrieval (top score and margin) on every response including successes.	The criterion it exists to prove is refuted. See §4.1. Also: it lives under platform/backend/gates/, not workshop/scripts/, and nothing in the shell control plane runs it — verify.sh scans only platform/gates/.
T081	FR-025 is enforced structurally : internal/api/router.go registers nothing answering and constructs nothing from a provider; the ask tree is mounted separately from pkg/answer/http.go. no_provider is an UnavailableCode, and pipeline_test.go:106 asserts the two enums are disjoint.	Neither named gate exists as an executable check. G-HTTP-5 appears once in the whole tree, in a comment (pkg/answer/http.go:24). G-CLI-13 appears five times, all in comments plus one test comment; provider_test.go:64 covers the server half (construction failure is not fatal) but nothing covers the start.sh half.
T083	The behaviour is built, and the live /api/ask/status.latency_note carries measured prefill/decode figures and an explicit “instant is off by two orders of magnitude”. estimated_seconds is null with a _basis string until measured (http.go:175-183, tested at http_test.go:144).	docs/answering.md does not exist. The honest statement lives in an API field and in docs/limits.md §1.4, not in the document the task names.
T095	docs/limits.md is the strongest document in the tree. It records SC-010 as not met (§1.1), L4 as a lexical floor (§1.2), the one-chapter corpus (§1.3), the latency reality (§1.4), and that declined is a correct result (§1.5) — with commands and outputs.	The SC-016a limit is absent — zero occurrences of SC-016a, “code-passage” or “code identity”. That limit is missing because the capability was never built (T014–T017), which is exactly when an honest-

Task	What exists	What is missing, and why it matters
		limits page most needs to say so.
T097	verify.sh V7 gate_pairing:445 requires every verify-*.sh to have a prove-*.sh, and --prove-failure includes a real-entry-point run over a mutated tree (:617-630).	Coverage is a fraction. V7 scans platform/gates/ only. platform/backend/gates/ is never scanned, and three of its four provers (prove-search-mutations.sh, prove-suggest-mutations.sh, prove-lumen-mutations.sh) are reachable from nothing — not from answering-gates.sh, not from verify.sh. No mechanism covers the Go_test.go gates at all.
T098	The three-valued discipline is real and paired-proven where it exists: verify.sh:799-812 (1 outranks 2), :636-650 (proof of exactly that), :652-663 (“a SKIP is never a PASS”), :772-773 (no gates found $\Rightarrow$ 2, never 0); answering-gates.sh:82-84 repeats the rule.	Not universal, for the same reason as T097 — the audit cannot reach what the aggregator cannot see.
T105	This document.	It reconciles the 120 tasks and reports on the success criteria. It does not trace all 45 FRs individually. That trace remains open.
T120	scripts/verify.sh is genuinely good: 7 static gates + runtime gate discovery, --prove-failure with ~20 prove_gate cases including controls that must stay green, the precedence rule implemented once and paired-proven in both directions, and an empty	It does not “run every gate in this contract set”. It never scans platform/backend/gates/, never runs the Go gates, and 22 of the 30 contracted gate identifiers do not exist anywhere in the tree (see §5.1). The aggregation mechanism is excellent; what it aggregates is a

Task	What exists	What is missing, and why it matters
	gate set treated as 2 rather than 0.	fraction of what the contract defines.

2.4 NOT BUILT — 44

Grouped by what their absence costs.

FR-039 (redaction) — the entire chain, 8 tasks: T012, T038, T039, T040, T076, plus the R-rule halves in T059 and T075 and the publish precondition in T087. Measured: zero files anywhere match `redaction*`; zero occurrences of `redaction-review` in any `.sh/.go/.py`; `INSERT INTO redactions` appears **zero times in the whole repository**. The `redactions` table DDL is declared (`registry.go:946-953`) and never written; `passages.redacted` is a bare column flip (`registry.go:1071`) — which T012 itself calls a contract violation. **There is no way to redact anything**. See §4.5, which is the most serious finding in this review.

Code-passage identity / SC-016a — 4 tasks: T014, T015, T016, T017. `pipeline/detect_symbols.sh` does not exist; P-U1 is unsettled and no evidence is recorded under `workshop/evidence/`. `symbol.go` and `ingest_match.go` do not exist. `symbol_aliases` DDL is declared (`registry.go:966-973`) and never written. G-PID-4 appears nowhere. Correctly, T015–T017 were blocked on T014 — the dependency was respected, which is why this is a clean gap rather than a mess.

The pipeline wrapper layer — 8 tasks: T005 (`venv-setup.sh`), T020 (`detect_backend.sh`), T023 (`reassemble.sh` wiring), T025 (`chunker`), T028 (`checkpoint/resume`), T032 (`pdf_notes.py`), T111 (`transcribe.sh`), T113 (`ingest.sh` as contracted). Whole-tree search returns **zero hits** for `transcribe.sh`, `index.sh`, `crossref.sh`, `redact.sh`, `add-chapter.sh`, `bench-suggest.sh`, `bench-search.sh`, `bench-retrieval.sh`, `detect_symbols.sh`, `detect_backend.sh`, `venv-setup.sh`. `ingest.sh` exists but parses only `--require-suspend`; none of `--write-anchors`, `--no-write-anchors`, `--kinds`, `--corpus`, `--dry-run`, `--check-idempotent` exists, and G-PID-6 (the double-run idempotency proof, I1–I6) is absent. No `chunker` exists — the real run was a **single unchunked 6 928.7 s pass** (`transcripts/full_ch01.runlog`, wall 4 854.79 s), so FR-029’s resumability was never needed and is unbuilt.

The `_common.sh` contract — 3 tasks: T106, T107, T119. `_common.sh` implements **0 of the 6** features T106/T107 name: no environment → config-file → live-probe → fallback ladder (only single-level `${VAR:-`

default}}, no (size, mtime, inode) source fingerprints, no self-hash re-check (script_modified_while_running appears only in the contract document), **no ERR or EXIT trap at all**, no evidence writer, no api_key_env. findings.jsonl exists nowhere. manifest.json/stdout.log/stderr.log are written by no script. Only verify-accuracy.sh:751 writes a result.json, and only on the measurement path. G-CLI-1, G-CLI-9, G-CLI-10 appear nowhere. FR-040 and SC-018 are unmet.

Retrieval measurement — 4 tasks: T064, T065, T067, T118. No retrieval.tsv and no retrieval benchmark file exists at any path. No bench-suggest.sh, no bench-search.sh, no bench-retrieval.sh, no latency evidence artifact. **SC-007 ($\geq 90\%$ top-5) and SC-008 ($\geq 80\%$ zero-overlap) are entirely unmeasured** — not failing, unmeasured, which is worse because nothing says so. docs/search.md (the media-boundary statement, FR-031) does not exist; docs/limits.md covers media but not that boundary explicitly.

US5 add-chapter — 6 tasks: T084, T085, T086, T087, T088, T090. add-chapter.sh does not exist, so none of the nine contracted stages, G-CLI-7, G-CLI-8, or the publish preconditions B1–B6 exist. **SC-011 is unmeasured.** Note that T088's stated rationale ("so idempotency can be proven without running ASR") is obsolete — see §6 — but the deliverable is absent regardless.

Accuracy measurement — 3 tasks: T035, T037, T041. No accuracy report and no accuracy.json exists anywhere; the live endpoint says measured: false. T035's specific mutation (inject a known WER, assert the scorer reports it) is not what --selftest does — that seeds a bluff into the reference guard, which is G-CLI-5's proof, a different assertion. T041's human speaker-attribution review has not been performed; the artifact carries **149 speaker unattributed** markers and the client asserts an unattributed speaker is never guessed (seek.spec.ts:185) — the refusal is built, the review is not. **SC-002 is unmeasured.**

Governance / close-out — 5 tasks: T022, T096, T099, T101, T104. T022 (review the passage contract *before* anything consumes it) never happened — consumers shipped first, and this document is the retrospective substitute. T096: scripts/check-registry.tsv contains **no workshop entries** (145 lines; the only match for "workshop" is a comment referencing specs/001), so verify-check-registry.sh cannot enforce SC-012/SC-013 on any of this feature's checks. T099: workshop/evidence/ holds exactly one file (phase2-passage-identity/README.md); the real evidence lives under platform/backend/evidence/ and platform/qa/evidence/ and is **uncommitted**. T101: CONTINUATION.md is stale — it states "specs/001-workshop-curriculum-platform/ is the only

feature directory” while `specs/002-*` exists with 7 artifacts, and describes the platform as “building has started ... nothing is committed”. T104: measured — **47 modified/untracked paths in workshop**, 5 in the umbrella including an untracked `specs/002-*`. Nothing has been committed or pushed.

Setup / misc — 3 tasks: T006, T114, T079. T006: `gofmt` is gated for exactly three packages (`answering-gates.sh:56-57`), not module-wide; **no ESLint configuration or lint script exists in the frontend at all.** T114: `G-CLI-15`, the containers-actually-booted anti-bluff, appears nowhere — so nothing asserts that a green `start.sh` implies the compose project’s containers are running and that `/api/health` was answered *through the published port*. T079: **no human certification of any citation is recorded anywhere**; SC-009 is unmeasured as specified.

2.5 SUPERSEDED — 0

No task was replaced by a recorded later decision. Two *premises* were superseded, and are recorded in §6 rather than here, because a stale premise in a plan is not a completed task.

2.6 CANNOT DETERMINE — 2

- **T102** — “Confirm workshop’s four governance carriers remain in lockstep (`verify-governance-cascade.sh C8`)”. All four carriers exist in `workshop/` at ~14.5 KB each, but lockstep is a byte-level property of a normalised comparison, and I did not run the verifier. Compounding it: `.gitmodules` now declares **13** gitlinks (the carriers describe 9), so the cascade’s own fleet derivation may or may not still classify correctly. **Not a pass.**
 - **T103** — “Run the whole gate suite green”. No recorded green run exists, and I did not run the suite (it mutates and it is heavy). Whether it *would* be green is undetermined; that it has not been *recorded* is determined. **Not a pass.**
-

3. What was built well

Named plainly, because the failures below are easier to act on when the good work is not lumped in with them.

1. **submodules/passage is the best-engineered thing here.** The four-outcome resolver keeps undetermined structurally distinct from `not_in_registry` rather than by convention. The prohibition list on `content_hash` (`registry.go:130-147`) enumerates four permitted and seven forbidden uses and the schema honours all of them. `LoadJSONL` uses `DisallowUnknownFields`, so an undefined key is *refused*, not silently dropped. And `pid_test.go`'s mutants are proven **discriminating** — `content-hash` keying must survive G2 and positional keying must survive G1 — which catches the failure mode where a mutation is so crude that every gate goes red and the proof means nothing.
2. **decoupling_test.go is a better test than it was asked to be.** It scans every `.go` file, `go.mod` and `README.md` for consumer vocabulary, checks exported types by reflection, checks the `Kind` type by AST, and checks the SQL schema by `PRAGMA table_info`. Its own header confesses that the older, narrower test “WAS the whole of the reusability claim” for two releases while the API carried a consumer’s vocabulary in a field, a tag, an enum and a column. That is the right instinct: strengthen the instrument and say what the old one missed.
3. **The three-state search contract is enforced at one construction point.** `NewEnvelope` (`envelope.go:302-510`) is the only way an envelope is built, and invariants I1–I9 are checked there with a test each. The measured consequence is visible from outside: a partial failure returns 503 with **no results key at all**, so a client cannot mistake it for emptiness.
4. **G-HTTP-8 is proven at the socket, not by mocking.** A real listener counts *accepted connections* across 200 suggest calls plus 25 deterministically-forced 503s. That is the difference between “we did not observe an embed call” and “an embed call was impossible”.
5. **verify.sh’s precedence rule.** 1 outranks 2, implemented in one place, paired-proven in both directions, with “no gates found $\Rightarrow$ 2, never 0” and “a SKIP is never a PASS” as separate proofs. Given that this repository has shipped seven pass/fail/could-not-determine conflations, getting this right once and proving it is worth more than the gate count.
6. **docs/limits.md volunteers its own bad news.** §1.1 states SC-010 is not met and names the fabrication count. §1.2 states the support

floor is lexical, not entailment. A document that says “the thing I document does not meet its criterion” is doing the job §11.4.6 asks for.

7. **The router’s unknown-/api handler, and the reasoning recorded with it.** A mounted SPA turned GET /api/search into a 200 text/html on a server implementing none of it — a probe checking only the status code would have certified an endpoint that did not exist. The fix registers GET /api/ → http.NotFound **with no contract body**, and the comment explains that the *absence* of a body is what lets a client tell “this route does not exist” from “this thing does not exist”. Verified live: unknown /api paths return 404 text/plain on all four verbs while real endpoints return 404 application/json with a contract body.
 8. **workshop-boot genuinely consumes the containers submodule.** It imports pkg/compose, pkg/health, pkg/logging, pkg/runtime and constructs the orchestrator with this project’s compose file — and it deliberately does **not** wrap cmd/boot, whose measured behaviour is to exit 0 having started nothing. The task warned about that trap and the implementation avoided it.
 9. **The QA challenge suite found both live defects before this review did.** B5 and B9 in platform/qa/challenges/api-challenges.sh assert exactly the two behaviours in §4.1 and §4.2, and platform/qa/evidence/prove/baseline.out:31,35 records both as [FAIL]. The instrument works. What did not happen is anyone acting on it.
-

4. What was built and is defective

4.1 above_floor: true on a score of 0.0 — RE-VERIFIED, reproduces

Status: confirmed, twice, on the live service.

```
GET /api/search?q=zzqxwvblorptfhgm          → 200  status=ok
  n_results 20 · above_floor true on 20/20 · every score exactly
0 · floor_calibrated false
GET /api/search?q=qqxzzzyvwmbhtrdplf        → 200  status=ok
  n_results 20 · above_floor true on 20/20 · every score exactly
0
```

Mechanism, pkg/search/service.go:485-495: s.Floor is 0 while uncalibrated, and the split is if d.Score < s.Floor — which is false for a score of exactly 0, so every hit takes the h.AboveFloor = true branch.

The code’s own comment says demoting on an unmeasured threshold “would be inventing a judgement”, and that reasoning is sound; the defect is that the *claim* `above_floor: true` is emitted anyway. A hit scoring 0.0 has cleared nothing.

Scope, measured per mode — this is narrower than “the default mode” and the difference matters:

mode	gibberish result
lexical	200 no_match — correct
code	200 no_match — correct
semantic	200 ok, 20 hits, all score 0, all above_floor: true
fused (the default)	inherits the semantic leg’s hits when that leg succeeds
bogusmode	400 unknown_parameter, field: mode — correct

So `no_match` is unreachable **specifically through the semantic leg**, and therefore through the default. The three-state contract is intact in `envelope.go`; what defeats it is that the semantic leg always returns *k* nearest neighbours and the floor never demotes them.

Already asserted and already failing: `platform/qa/challenges/api-challenges.sh:196-209` (challenge B9), recorded [FAIL] B9 ... zero-score-but-above_floor=20 of 20 hits; `floor_calibrated=False` in `platform/qa/evidence/prove/baseline.out:35`.

Note for whoever fixes it: the envelope already has the right vocabulary. `near_misses` exists, invariant I6 requires `above_floor: false` on every near-miss, and I7 requires a `no_match` built on an uncalibrated floor to say `floor_calibrated: false`. The parts are there; they are not wired to the uncalibrated case.

4.2 Unknown query parameters silently accepted — RE-VERIFIED, reproduces

Status: confirmed.

```
GET /api/search?q=x&nosuchparam=1      → 200  status=ok, 20
results                               ← silently honoured
GET /api/suggest?q=x&bogus=2           → 200
status=ok                             ← silently honoured
GET /api/search?q=x&limit=notanumber  → 400  unknown_parameter,
```

```
field=limit
GET /api/search?q=x&mode=bogusmode    → 400  unknown_parameter,
field=mode
GET /api/search                        → 400  empty_query, field=q
```

The closed §5.2 enum **has** the word for this fault, and the endpoint uses it — but only for a bad **value** of a known parameter, never for an unknown **name**. `internal/api/search.go` reads parameters positively (`r.URL.Query().Get("q"), ...Get("mode"), ...Get("kinds"), ...Get("chapter")`) and never enumerates what it did *not* consume, so an unrecognised name cannot be detected. The caller gets a window they did not ask for and cannot tell.

Already asserted and already failing: `api-challenges.sh:159-168` (challenge B5), recorded [FAIL] B5 ... OBSERVED: `http=200`
`ct=application/json code=None in platform/qa/evidence/prove/`
`baseline.out:31.`

A third, adjacent finding surfaced while probing this: `?limit=999999` returns **200 with 100 results** — the value is silently clamped, with no error and no field in the response saying it was clamped. Same failure shape as B5: the caller's instruction was altered without being told.

4.3 SC-010 fails on the live service — an unanswerable question was answered

This was not on the known-defect list and it is the most consequential runtime finding.

Recorded evidence, `platform/backend/evidence/answering/`:

Run	A/ OK	A/ DECLINED	U/ REFUSED	U/ FABRICATED
bench-2026-09-01.tsv	4	4	8	2
bench- expanded-2026-09-01.tsv	17	7	30	3

Re-verified live tonight, one class-U question from the expanded set (an uncomputable aggregate — the corpus states a monthly figure and never an annual one, so answering requires arithmetic the passages do not contain):

```
POST-equivalent GET /api/ask?q=... → job accepted
  final: status = answered · citations = 2 · text present
  retrieval: top_score 0.7244 · margin 0.2334 · min_score 0.45 ·
```

```
min_margin 0.05
      retrieved 8 · admitted 8
```

It still fabricates. SC-010 requires ≥ 10 unanswerable questions declined **100%** with **0** fabricated. Measured: 3 of 33.

Why L1 cannot catch this: the retrieval gate is working exactly as designed — top score 0.72 against a floor of 0.45, margin 0.23 against 0.05. The passages *are* topically relevant. The question is unanswerable not because retrieval is weak but because the answer requires a computation the corpus does not support, and **only L4 can catch that** — which brings us to:

4.4 L4 is disabled on the deployed server, and the server says so

```
cmd/workshop-server/main.go:475-494 calls answering.Wire without
EntailPython or EntailModelDir. Per internal/answering/
wire.go:141-147, unset paths mean L4 falls back to the lexical floor.
Only cmd/workshop-ask exposes -entail-python / -entail-model flags.
```

The live service reports this itself. Its `support_verification` field, returned by `GET /api/ask/status` and defined as a string literal in `workshop/platform/backend/pkg/answer/verify.go`, states in substance — **paraphrased, not quoted, because that file is in the PRIVATE submodule** — that the check is a lexical content-word overlap test, which a claim clears once at least half of its content words also occur somewhere in whatever passages back it; that this is a NECESSARY condition for support and explicitly NOT entailment; that it cannot separate a claim the passage actually states from one that merely reuses its vocabulary; that a claim scoring a perfect 1.00 can still invert the passage’s meaning via a single inserted negation; and that this layer is what the system drops back to whenever no entailment model can be reached.

(Read the field itself at that path for the exact wording. It is quoted here **by path only** — this document is in the PUBLIC umbrella and `workshop/` is PRIVATE, so reproducing the literal would be a disclosure, per the standing rule in `docs/content-boundary.md`.)

That is an admirably honest field. It is also a precise description of how §4.3’s fabrication gets through: a claim that multiplies a stated monthly figure shares nearly all its content words with the passage that states it.

The entailment verifier itself is real and was genuinely exercised — pkg/entail/onnx.go (ONNX int8 cross-encoder, model identity read from the **loaded** files rather than a compiled-in constant), with eleven dated evidence artifacts under evidence/answering/. It is built and not deployed. Note also that **every model-touching entail test t.Skips without env vars** (onnx_test.go:23, model_identity_test.go:41, eval_test.go:92), so go test ./... completes pkg/entail in 15 ms without exercising the model — a green suite that proves nothing about the layer.

4.5 FR-039 is decorative, and Chapter 1 is being served without its review

The plan said this in as many words: “*Removing any one of them leaves FR-039 decorative.*” All of them are missing.

Measured: - Zero files match redaction* anywhere in the tree. - Zero occurrences of redaction-review in any .sh, .go or .py. - **Zero INSERT INTO redactions** in the whole repository. The table DDL exists and is never written. - passages.redacted is a bare column flip from boolInt(rec.Redacted) (registry.go:1071) — the exact thing T012 calls a contract violation. There is no unredact, so no history.

Consequently: - **redaction-review.json does not exist for Chapter 1**, and T040 forbids export, publication, serving or committing before it does. - The transcript **is committed** in the workshop submodule. - The chapter **is served live** on the LAN at 192.168.1.44:8087. - **GET /api/chapters discloses the raw material filenames**, and those filenames identify the third party whose presence is the stated reason FR-039 exists. The transcript endpoint serves 1 055 passages of their speech.

The publish precondition that would have blocked this (B5 in T087) lives in add-chapter.sh, which does not exist. internal/api/recording_test.go:767 asserts that *recording responses* never leak the filename — so the concern was understood in one place and missed in the adjacent one.

This is a governance finding, not a code-style one. It is the single item in this review that should be actioned before anything else in the ledger.

4.6 The coverage identity is failing and nothing is acting on it

GET /api/chapters/01/transcript returns, live:

```
coverage: complete = false
          duration_s      6928.713
          speech_span_s    6545.98
          accounted_silence_s 0
          unexplained_gap_s 382.733      ← 5.5 % of the runtime
          gaps: [ ... each with reason "unmeasured" ... ]
```

The arithmetic is right and the endpoint is honest. But `accounted_silence_s` is **0** because the VAD output (`audio_energy.py`, T024) is never fed into the coverage computation — the two halves were built and never joined. So 382.7 seconds of the recording are unaccounted for, `complete` is false, and **B1** (`coverage.unexplained_gap_s == 0`) would block publication if the publish gate existed. It does not, so the chapter is served anyway. SC-001 is not met.

4.7 Every registry row claims an anchor mode the files do not carry

Measured:

```
passages.jsonl: 1101 rows, all with source_ref.anchor = "inline"
curriculum/chapter-01/transcript.md : grep -c 'pid:' → 0 ; no
'<!--' lines at all
curriculum/chapter-01/exercise-01.md : grep -c 'pid:' → 0
```

The registry declares that identity travels *inline, in the source file*. It does not. Identity exists only in `passages.jsonl` and the derived `.db` — i.e. `AnchorRegistryOnly` in the contract's vocabulary, recorded as `inline`.

Two consequences, one measured and one not: - **Measured**: `ingest.sh` has no `--write-anchors` flag, so nothing ever writes them. - **Not measured, and I will not assert it**: whether a re-ingest attaches or re-mints without anchors depends on which Sync branch fires, and I did not run `ingest`. What is certain is that the anchor-attach path (R1c: an anchor present but not in the registry is a hard error) can never fire on this corpus, so the survival guarantees G1/G2 — which are proven, rigorously, on synthetic files *with* anchors — are unexercised against the real one.

4.8 Two progress stores that never meet

`core/progress.ts` writes `localStorage` only; `features/progress/progress.component.ts:190` reads `GET /api/progress`, which is registered (`main.go:303-304`) and live (400 without X-Session). The transcript reader's position is therefore invisible to the progress page. The comment in `progress.ts` justifying the `localStorage` choice cites a measured 404 on `/api/progress` — that measurement is stale.

4.9 The a11y suite has never passed

`e2e/artifacts/results.json (2026-09-01T21:21Z):` expected: 20, unexpected: 58, every failure `net::ERR_CONNECTION_REFUSED`. The suite is well written — axe per route in both themes, plus real ARIA 1.2 combobox keyboard handling — and it has produced no green run. SC-017 is unmeasured, and the artifact currently on disk could be mistaken for a run that found problems rather than a run that never happened.

4.10 The semantic leg is intermittently unavailable

Across roughly 20 live probes tonight, `mode=semantic` and `mode=fused` alternated between answering and returning 503 `partial_failure_zero_results` with `legs.semantic = "failed"`. At one point four consecutive retries over ~30 s all failed while `mode=lexical` answered every time. The degradation is **honest** — that is the three-state contract working, and it is the right behaviour — but a search surface whose primary leg fails under light probing load will not meet SC-006 in use. This is consistent with the tuning note already recorded in the repository about embedding serialisation under `OLLAMA_NUM_PARALLEL=1`.

4.11 Only two files in the entire tree reach the platform

`ingest.sh:271-287` hardcodes exactly two sources per chapter — `transcript.md` and `exercise-01.md`. Measured composition of `passages.jsonl`:

```
1101 passages: transcript_segment 1055 · doc_section 44 · code 2
by source path: 1055 from chapter-01/transcript.md
                 46 from chapter-01/exercise-01.md
```


Everything else is invisible to search and to answering: all 17 documents under `workshop/docs/` (including the five training-area documents, the user guide, the manual, the FAQ and `limits.md` itself), and every file under `curriculum/chapter-01/knowledge/` (areas, terms, lexicon, artifacts, question banks, coverage, reverse index, linkage). `md_sections.py` also drops sections shorter than `MIN_CHARS = 120` silently — 48 headings in `exercise-01.md` produced 44 `doc_section` passages — which is a defensible rule, but the drop is not reported.

The `--corpus` flag that decision D2 relies on (“the corpus is the `workshop/ module` **plus the vasic monorepo**”) does not exist, which is why the whole monorepo contributes exactly 2 code passages. Any question whose answer lives in `docs/` is structurally unanswerable, and the system cannot tell the user that.

5. What was built that no task asked for

Not a complaint — some of it is the best work here. But it is unbudgeted, unreviewed against any spec, and in one case it changes the application’s front door.

5.1 A different gate vocabulary from the one the contracts define

The contracts define **30** gate identifiers (`G-CLI-1..17`, `G-HTTP-1..8`, `G-PID-1..6`). Measured occurrences in implementation code across `workshop/` and the two submodules:

Present: `G-CLI-5`, `G-CLI-11`, `G-CLI-13` (comments only), `G-CLI-16`, the whole `G-HTTP-1 ... G-HTTP-4` block, `G-HTTP-5` (one comment), `G-HTTP-6 ... G-HTTP-8`, and `G-PID-3`.

Absent entirely: `G-CLI-1`, 2, 3, 4, 6, 7, 8, 9, 10, 12, 14, 15, `G-PID-1`, 2, 4, 5, 6.

Invented instead, and not in any contract: **`G-CLI-17`** (a stopped stack must be a determined negative distinguishable by exit code — with a prover that compiles, vets **and links** its mutations into the real `workshop-boot` before requiring the gate to go red), **`G-IDX-1`**, **`G-ANS-BUILD/VET/FMT/UNIT/RACE/MUTATE/BENCH`**, **seven `G-LUMEN` gates**, **nine `G-SUG` gates**, **`G-WEB-1`**, **`G-DECOUPLE`**, and the QA challenge families `A*/B*/C*`.

The invented gates are, on the whole, good — G-LUMEN-2 (a glued symbol never reaches a renderable field) and G-SUG-8 (a material suggestion's target actually resolves against the endpoint that serves it) are exactly the kind of check that catches real bugs. But the contracts' closure condition in `tasks.md` §“Gate coverage” — *“a gate defined in a contract and built by no task is a gate that will not exist”* — is now false for 17 of 30, and the substitution was never recorded.

5.2 A whole knowledge layer, built ahead of the spec that asks for it

`workshop/curriculum/chapter-01/knowledge/` contains `areas.json`, `terms.json`, `lexicon.json`, `artifacts.json`, `coverage.json`, `reverse-index.json`, `linkage.jsonl`, `platform-mapping.json`, two question banks, `build.py`, `verify.py`, a `TAXONOMY.md` and a `README.md`. The frontend carries `core/knowledge.ts` (537 lines) and **five feature directories nobody asked for in 001** — `features/{areas,practice,ask,progress,curriculum}` — and `/api/suggest` already returns `area_code`, `area_slug`, `lesson_slug` and `sid` fields in its suggestion targets.

This is spec 002's subject matter (`specs/002-knowledge-areas-deep-linking/`, 7 artifacts, 2026-09-01), built before 002's execution phase began. The header comment at `workshop/platform/frontend/src/app/core/knowledge.ts:12-18` is honest about its own status — **paraphrased, not quoted, because that file is in the PRIVATE submodule**: it records a 404 from each of the two endpoints it depends on (`/api/areas`; separately, `/api/quizzes`) on that build, and concludes that the type shapes declared beneath it amount to a STATED CONTRACT the client programs against rather than an observation of any working endpoint. (Exact wording at that path; cited **by path only**, per `docs/content-boundary.md`.)

One consequence deserves a decision rather than a shrug: `app.routes.ts:31-33` routes `'` to `CurriculumComponent`, so **/ is no longer the chapter list**. Feature 001's US2 independent test is *“start with the documented command, confirm Chapter 1 is listed”* — the front door moved to serve a feature that has not been executed yet.

5.3 An upstream-contributions staging area

`workshop/platform/upstream-contributions/rag-grounding/` is its own Go module with `grounding.go`, `extractive.go`, `provider.go` and a test — code staged for contribution back to the RAG submodule rather than

forked in place. That is the right instinct and it is not in the plan. Its README.md:34 is also the tree's own honest record that loopback egress refusal is **ABSENT**.

5.4 Two new public submodules

.gitmodules now declares **13** gitlinks, not the 9 the umbrella carriers describe. submodules/passage and submodules/verdict were extracted as separate published repositories — which tasks.md records as a correction (the “FILE PATHS CORRECTED 2026-09-01” block) — and submodules/LLMProvider and submodules/RAG were added and are consumed by the backend (go.mod:86-92). The extraction is exactly what the plan's own reasoning about pkg/ vs internal/ argues for. The carriers not being updated to match is a separate, small debt.

5.5 A QA challenge suite with mutation provers

platform/qa/ — api-challenges.sh, answering-challenges.sh, state2-challenges.sh, each with a prove-*.sh, plus a mutating-proxy.py and a _bank.sh. It is the instrument that already found both known defects. Nothing in the plan asked for it, and nothing in the control plane runs it (verify.sh scans only platform/gates/).

5.6 A second ASR engine, built and calibrated

pipeline/engines/whisper.cpp is vendored and **built** (build/bin/whisper-cli, 2.4 MB, plus whisper-server, whisper-bench, whisper-quantize), with 874 MB of GGML weights alongside the CTranslate2 weights. compare_engines.py implements a word-level Levenshtein alignment and — the part worth keeping — measures how often engine B contradicts engine A's *high-confidence* words, explicitly refusing to treat agreement as correctness. The plan named one engine. Two were built and diffed.

6. Stale premises found while reconciling

Recorded because the plan and the carriers still assert them, and an agent reading either would act on false facts.

Assertion, and where	Measured today
CLAUDE.md: “No ASR engine ... whisper, faster_whisper, vosk ... all absent”	False. pipeline/venv/bin/python -c "import faster_whisper, ctranslate2" → OK 1.2.1 4.8.2, and whisper.cpp is built with weights on disk. The system python3 is indeed bare — the venv is mandatory, which detect_media.sh:206-209 correctly checks for.
CLAUDE.md and tasks.md Phase 6: “no generative model exists on this host ... both are embedding models”	False. Live /api/ask/status reports enabled: true and calibrated: true for model: qwen2.5:3b-instruct-q4_K_M behind provider: ollama. Phase 6’s “BLOCKED ON AN OPERATOR ACTION” banner is obsolete.
tasks.md File Structure: “questions.tsv currently holds 8 A rows and 10 U rows ... 8 is not 20, and the shortfall is real”	Superseded. Measured: 24 A, 33 U . SC-009’s count requirement is met; SC-010’s count requirement is met. The <i>shortfall</i> is now in certification and outcomes, not in counts.
CLAUDE.md: “.gitmodules declares nine gitlinks”	False. Thirteen.
CONTINUATION.md: “specs/001- workshop-curriculum-platform/ is the only feature directory”	False. specs/002-knowledge- areas-deep-linking/ exists with 7 artifacts.
core/progress.ts header: /api/ progress measured 404	False now. Registered at main.go:303-304; live GET /api/ progress → 400 X-Session is required.
tasks.md T088 rationale: “otherwise every US5 proof inherits the ASR block”	Obsolete — there is no ASR block. The deliverable is still absent.
route-manifest.tsv “DEBT=4”	Not four missing endpoints. All four NOT_BUILT rows are /api/ nosuchroute on GET/POST/PUT/ DELETE — negative controls asserting an unknown path returns a plain 404. All four behave as declared (verified live:

Assertion, and where	Measured today
	404 text/plain). Reading “DEBT=4” as four unimplemented contract routes would be wrong. The manifest does, separately, omit POST /api/ask and POST /api/progress, both of which are registered and live.

7. Success criteria — measured, failing, or unmeasured

SC	State	Evidence
SC-001 coverage identity closes	FAILING	live unexplained_gap_s: 382.733, complete: false (\$4.6)
SC-002 transcript accuracy over ≥ 30 windows	UNMEASURED	/api/chapters/01/accuracy → measured: false; no accuracy.json exists
SC-003 timestamp within 5 s of spoken content	UNMEASURED as written	seek.spec.ts proves the <i>UI seek</i> is exact and < 5 s; the criterion is about transcript timing, which needs SC-002’s reference
SC-004 fresh clone → running in 15 min	UNMEASURED	claimed in docs/quickstart.md:3, not timed; venv-setup.sh absent
SC-005 suggest ≤ 200 ms p95	MET, ad hoc	10 live calls at the HTTP boundary: p50 9.8 ms, p95 10.3 ms, max 11.0 ms. The contracted harness (bench-suggest.sh) does not exist

SC	State	Evidence
SC-006 search ≤ 2 s p95	UNMEASURED	no harness; and see §4.10
SC-007 $\geq 90\%$ top-5	UNMEASURED	no retrieval benchmark exists at any path
SC-008 $\geq 80\%$ zero- overlap	UNMEASURED	same
SC-009 100% of citations support their claim, ≥ 20 answers	UNMEASURED	24 A rows exist; 17 answered in the last run; no human certification recorded anywhere
SC-010 ≥ 10 unanswerable, 100% declined, 0 fabricated	FAILING	3 fabricated of 33 recorded; re-verified live tonight (§4.3)
SC-011 new chapter in <30 min, zero code change	UNMEASURED	add-chapter.sh does not exist
SC-012 100% of checks carry a paired failure proof	PARTIAL	enforced for platform/gates/ only; three provers under platform/backend/ gates/ are reachable from nothing
SC-013 100% distinguish could- not-determine	PARTIAL	rigorous where implemented (verify.sh:799-812, answering- gates.sh:82-84); not universal
SC-014 zero active server-side CI	MET	git -C workshop ls- files $\rightarrow$ no workflow files; the 29 vendored whisper.cpp workflow files are git-ignored and correctly classified out-of-scope by verify.sh V5, which prints them rather

SC	State	Evidence
SC-015 pushed and verified with <code>git ls-remote</code>	NOT MET	than suppressing them 47 uncommitted paths in workshop, 5 in the umbrella
SC-016 pid survives correction and movement	MET	<code>pid_test.go</code> G1/G2 with discriminating paired mutations — but proven on synthetic anchored files, and the real corpus carries no anchors (§4.7)
SC-016a code-passage identity	NOT IMPLEMENTABLE as specified	P-U1 unsettled; T014 never ran
SC-017 WCAG 2.1 AA, zero A/AA violations	UNMEASURED	suite written, never passed (§4.9)
SC-018 evidence for every run	NOT MET	no evidence writer exists; <code>findings.jsonl</code> appears nowhere
SC-019 each component builds with the curriculum absent	PARTIAL	<code>decoupling_test.go</code> proves no consumer <i>vocabulary</i> — stronger in one dimension. No test compiles or runs any package with the consumer removed

8. What this review recommends becomes a task, not a patch

In priority order. None of these was implemented here; that is the point of the phase.

1. **FR-039 first.** Either build the redaction chain (T012, T038, T039, T040) before the platform serves Chapter 1 again, or record an

operator decision that the review is “none required” — an explicit decision is valid; the absence of one is not. Right now a third party’s speech and name are served on the LAN and committed to a submodule with no recorded review. This is the one item that should not wait for the rest of the ledger.

2. **Make no_match reachable.** Wire the uncalibrated case to `near_misses + above_floor: false`, which the envelope already supports (I6, I7). Do not calibrate a floor by guessing — emitting `above_floor: true` on a zero score is the bug, not the absence of a threshold.
 3. **Reject unknown parameter names.** Enumerate the consumed set and refuse the remainder with the `unknown_parameter` code the enum already provides. Fold in the silent `limit` clamp.
 4. **Deploy L4’s entailment, or state in docs/limits.md that the deployed server runs the lexical floor.** `main.go` needs to pass `EntailPython/EntailModelDir`. Until it does, SC-010 cannot be met — §4.3 and §4.4 are one defect seen from two ends.
 5. **Join the coverage halves.** Feed `audio_energy.py`’s silence spans into `ComputeCoverage` so `accounted_silence_s` stops being 0 and SC-001 becomes answerable either way.
 6. **Decide the detect_media.sh exit semantics** (T019) — 1 or 2 for an unusable tool. This is a contract question, not a code question.
 7. **Widen the corpus** (`--corpus`, T113) so `docs/` and `knowledge/` reach the index, or state in `limits.md` that they do not.
 8. **Point verify.sh at platform/backend/gates/** and at the Go gates, or record why it should not. Three mutation provers currently run only if someone types their path.
 9. **Register this feature’s checks** in `scripts/check-registry.tsv` (T096) so SC-012/SC-013 can be enforced mechanically rather than read off this document.
 10. **Reconcile the gate vocabulary** (§5.1): promote the invented gates into the contracts, or build the 17 contracted ones, or record the substitution. The closure condition in `tasks.md` is currently false and silently so.
 11. **Refresh the stale premises in §6** — the ASR and generative-model claims in `CLAUDE.md` and in `tasks.md` Phase 6 will otherwise keep an agent from doing work that is already possible.
 12. **Decide whether / should be the chapter list** (§5.2) before feature 002 executes.
-

9. Honest boundary on this review itself

- I did not run `verify-governance-cascade.sh`, nor `scripts/pre-push-gates.sh`, nor `verify.sh`, nor `verify.sh --prove-failure`. T102 and T103 are therefore **CANNOT DETERMINE**, not passes.
- I did not run `ingest.sh`, so §4.7's consequence for re-ingest is stated as unmeasured rather than asserted.
- I did not stop ollama, so G-HTTP-5's and G-CLI-13's runtime assertions were not exercised; their absence from the tree is a code-search result, which is sufficient for the verdict.
- `go test ./...` in `platform/backend` was run and passed (14 packages ok, 0 FAIL) — with the caveats that `pkg/curriculum` (690 lines) and `pkg/embed` (249 lines) have **no test files at all**, and `pkg/entail` completes in 15 ms because every model-touching test skips.
- All live measurements were taken against `192.168.1.44:8087` between 06:10 and 06:40 UTC on 2026-09-02, at index generation 21, `pid_count` 1101. The semantic leg's availability varied during that window (§4.10); every conclusion drawn from it is stated with the number of observations behind it.
- **The tree moved while this review was being written.** Between the start of the inventory and its close, another session created `workshop/pipeline/{extract,mentions,authoring}/`, `workshop/platform/backend/pkg/{knowledge,assessment}/`, `workshop/platform/frontend/src/app/features/plans/`, `workshop/scripts/prove-export-capabilities.sh` and `docs/content-boundary-incident-2026-09-01.md` — the count of changed paths in `workshop` went from 47 to 55 during the session. None of those paths is a feature-001 deliverable, so no verdict above is invalidated by them, but **every “NOT BUILT” here is a statement about the tree as of 2026-09-02 ~06:40 UTC and must be re-derived before it is acted on.** Re-derive with the one-liners cited in each finding rather than trusting this document — which is the same rule this document applies to every other document in the repository.
- No fix was applied. Nothing was committed or pushed. Files written: exactly `specs/001-workshop-curriculum-platform/tasks.md` (54 checkbox flips, no other change — verified with `git diff`), `specs/001-workshop-curriculum-platform/review.md` (new), and an appended section in `.superpowers/sdd/progress.md`.